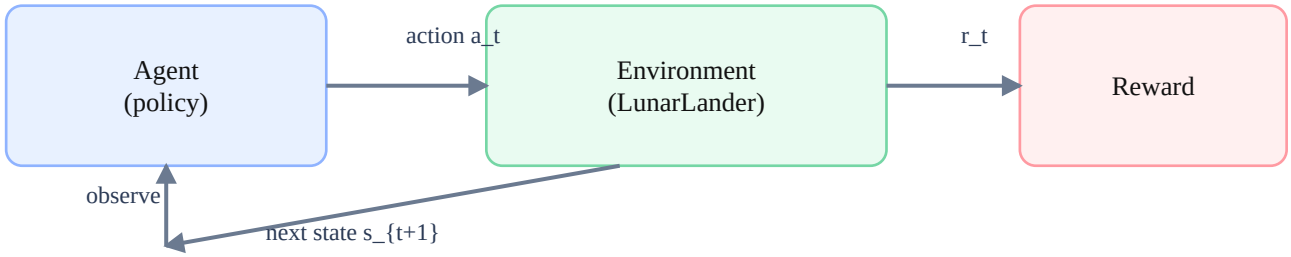
Reinforcement Learning for LunarLander (PPO)

A visual, beginner-friendly guide for a portfolio web page

Goal: explain what you did (PPO on LunarLander) so that a reader with *zero* RL background can follow, while still showing the key maths behind it.

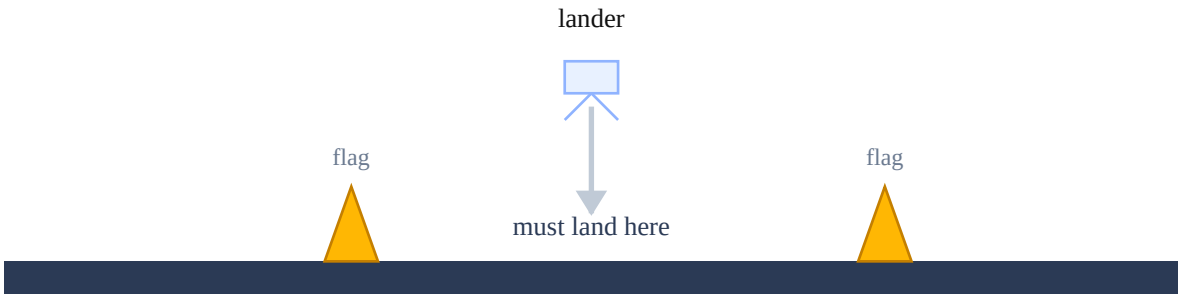
One step in RL



How to read this guide: each equation appears in a grey box, and right below it you get a plain-English translation.

1. The task in one picture

LunarLander is a small simulation where an agent controls a spacecraft. At every time step it chooses one of 4 discrete actions: fire left engine, fire right engine, fire main engine, or do nothing. The objective is to land between two flags with low speed and without crashing.



2. The RL vocabulary (state, action, reward)

Think: 'What I see' → 'What I do' → 'What I get' → 'What I see next'

State = what the agent observes now (position, velocity, angle, etc.). **Action** = what the agent decides to do. **Reward** = a number that says how good that step was. The agent learns by trying actions and using the rewards to adjust its policy.

3. What the agent is trying to maximize

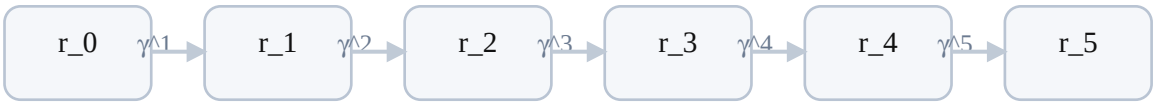
In RL we do not optimize 'accuracy'. We optimize **total reward over time**.

Equation (Return):

```
G = r_0 + γ r_1 + γ^2 r_2 + γ^3 r_3 + ...
```

Translation: total score = reward now + (gamma × next reward) + (gamma² × later reward) + ...
Gamma γ is between 0 and 1. If γ is close to 1, the agent cares a lot about the future.

Total score = rewards now + discounted future rewards



γ (gamma) close to 1 ⇒ future matters a lot

4. Policy network: the agent's 'brain'

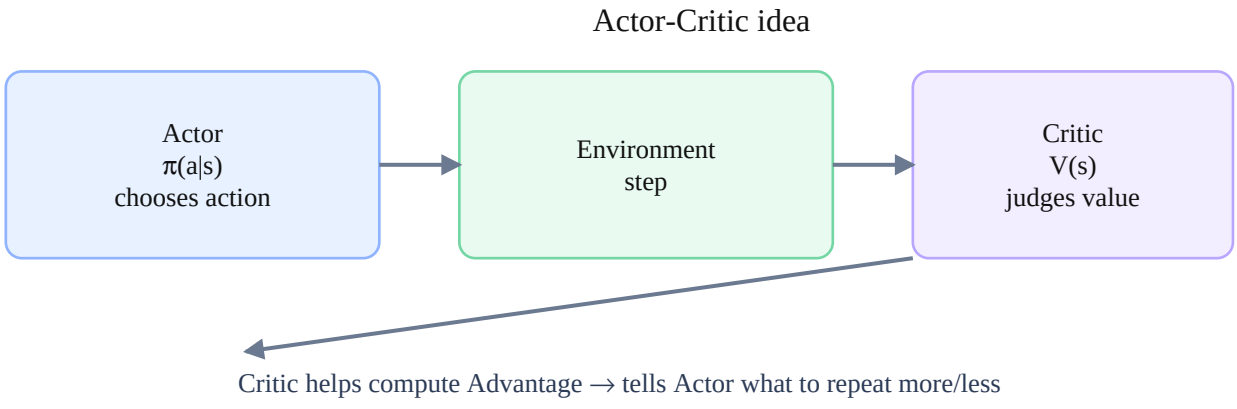
A **policy** is a rule that maps states to actions. With a neural network, the output is usually a **probability** for each action.

Equation (Policy):

```
π(a | s) = probability of choosing action a when we are in state s
```

Translation: the policy says: 'in this situation, how likely is each action?' During training we adjust the network so that good actions become more likely.

5. Actor-Critic: two neural networks working together



Actor = the policy network (chooses actions). **Critic** = a value network that predicts how good a state is. The critic helps reduce randomness and makes learning more stable.

Equation (Value function):

```
V(s) ≈ expected future return starting from state s
```

Translation: the critic is like a coach saying: 'from here, I expect you will score about X if you keep playing well.'

6. Advantage: did this action do better than expected?

This is the key training signal for PPO.

Equation (Advantage):

```
A(s,a) = (what actually happened) − (what the critic expected)
```

Translation: if Advantage is positive, the action was surprisingly good → do it more. If Advantage is negative, the action was worse than expected → do it less.

7. Why PPO? The 'policy collapse' problem

In RL, today's data depends on today's policy. If we update the policy too aggressively, the agent can suddenly behave very badly, collect bad data, and training can collapse.

Mental model:

Learning is like improving your driving by small adjustments. If someone forces you to change everything at once, you may get worse.

8. PPO's solution: clip updates so changes stay small

PPO compares the new policy to the old policy using a ratio. Then it **clips** the ratio so the update cannot be too big.

Equation (Policy ratio):

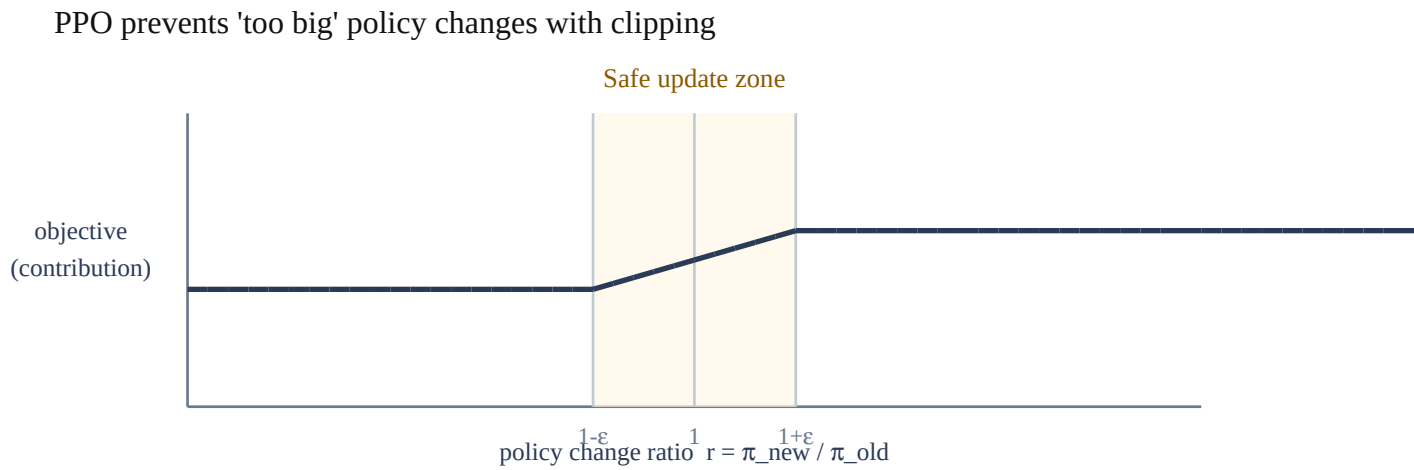
```
r_t(θ) = π_θ(a_t | s_t) / π_{θ_old}(a_t | s_t)
```

Translation: this ratio tells us how much more (or less) likely the new policy is to take the same action, in the same state.

Clipping rule:

```
clip(r, 1−ε, 1+ε) keeps r inside [1−ε, 1+ε]
```

Translation: PPO allows learning, but blocks extreme policy changes. This is the main reason PPO is stable and popular.



9. Hyperparameters (what they mean in human language)

Hyperparameter	What it controls	Plain-English intuition	Your value (example)
Learning rate (actor/critic)	Size of network updates	Bigger steps learn faster but can ‘break’ training	0.001 / 0.001
Gamma (γ)	How much future rewards matter	Close to 1 means long-term planning (landing safely)	0.99

Lambda (λ)	Smoothing for advantage estimates (GAE)	Reduces noisy learning signals; smoother training	0.9
Entropy coefficient	How much the agent explores	Encourages trying different actions early on	0.2
Clip (ε)	Max policy change per update	Prevents giant updates; keeps training stable	0.4 (often 0.1–0.3 is common)

Portfolio tip: add one plot (reward over episodes) and a short GIF/video of the trained lander. The combination of a clean diagram + a demo video usually gets the most attention.